REPORT: Cryptographic Implementation Analysis

Comparative Analysis of AES-128 and ChaCha20 Algorithms

Date: August 3, 2026
Name-: Abhijit Santra
Roll No-: CRS2502

---

1. Executive Summary

This report provides a detailed comparative analysis between two fundamental cryptographic primitives: the **Advanced Encryption Standard (AES-128)** and the **ChaCha20** stream cipher. Based on low-level C implementations, this document evaluates their architectural differences, underlying mathematical models, CPU clock cycle performance, and optimal deployment scenarios.

While AES remains the undisputed enterprise standard, ChaCha20 has emerged as the premier alternative for environments lacking dedicated cryptographic hardware. This report outlines exactly *why* and *when* to choose one over the other.

2. Core Architectural & Mathematical Differences

The fundamental difference between AES and ChaCha lies in their mathematical architecture. AES relies on a **Substitution-Permutation Network (SPN)**, whereas ChaCha utilizes an **ARX (Add-Rotate-XOR)** network.

2.1 AES-128: The Galois Field Approach

AES treats data as a 4×4 matrix of bytes (16 bytes / 128 bits total). Its security relies heavily on complex byte-level algebraic transformations over the finite Galois Field GF(2^8).

- **Non-Linearity (S-Boxes):** AES achieves cryptographic confusion using pre-calculated Lookup Tables (S-boxes). Every byte is substituted based on its mathematical inverse in GF(2^8).
- **Diffusion (MixColumns):** AES multiplies the state matrix by a fixed polynomial matrix using GF(2^8) math. This ensures that altering a single input bit affects all output bits.
- **Memory Dependency:** Because AES uses lookup tables, software-only implementations require continuous memory access during encryption rounds, making them theoretically vulnerable to cache-timing side-channel attacks.

2.2 ChaCha20: The ARX Approach

ChaCha20 is a stream cipher that operates on a much larger 4×4 matrix of **32-bit words** (64 bytes / 512 bits total). It abandons complex algebraic fields in favor of native CPU instructions.

- **The ARX Design:** ChaCha achieves security using only three simple operations:
  1. **A**ddition (Modulo 2^{32})
  2. **R**otation (Bitwise circular shifts)
  3. **X**OR (Bitwise exclusive OR)
- **No Lookup Tables:** ChaCha calculates everything dynamically. It does not use S-boxes.
- **Mathematical Simplicity:** By relying on standard integer addition and bitwise logic, ChaCha operates entirely within CPU registers. This mathematically guarantees immunity to memory-cache timing attacks.

3. CPU Clock Cycle & Performance Analysis

When benchmarking the implementations using the hardware `__rdtsc()` intrinsic, the performance disparity between the two algorithms becomes heavily dependent on the host hardware.

3.1 Software-Only Execution (Without Hardware Acceleration)

When compiling the raw C code without enabling hardware intrinsics, ChaCha drastically outperforms AES.

Algorithm	Base Operation	Cycles per Byte	Bottleneck
AES-128	8-bit Field Math	∼15 to 20	S-box memory lookups & GF(2^8) math.
ChaCha20	32-bit ARX Math	∼3 to 4	None. Maps perfectly to CPU registers.

Why ChaCha Wins in Software

A 32-bit CPU can process ChaCha's 32-bit additions in a single clock cycle. Conversely, calculating Galois Field multiplication in software (or fetching from an S-box in RAM) takes multiple clock cycles. In a pure software race, **ChaCha is roughly 3x to 5x faster than AES.**

3.2 Hardware-Accelerated Execution (AES-NI / ARMv8 CE)

Modern desktop and server CPUs (Intel, AMD, Apple Silicon) contain dedicated silicon specifically engineered to calculate AES. This is known as **AES-NI** (New Instructions).

- **The AES-NI Advantage:** When the compiler maps AES directly to hardware, the complex GF(2^8) math is bypassed. The CPU executes a full AES round in a single hardware tick.
- **Accelerated Ticks:** With hardware acceleration, AES drops from ∼18 cycles per byte to **∼0.5 to 1 CPU cycle per byte**.
- **SIMD ChaCha:** ChaCha can also be accelerated using AVX-512 or NEON vector instructions, processing multiple blocks simultaneously, bringing it down to **∼1 to 1.5 cycles per byte**.

3.3 Pipeline Parallelism

- **AES (CBC Mode):** Highly sequential. Block 2 cannot be encrypted until Block 1 is finished. This creates pipeline stalls in the CPU architecture.
- **ChaCha20:** Naturally parallel. Because it is a stream cipher (essentially operating in a counter mode), Block 1, Block 2, and Block 100 can be computed simultaneously across multiple CPU cores.

4. Strategic Deployment: Which is Better and When?

Both algorithms provide a cryptographic security margin that cannot be broken by modern (or near-future quantum) computers. Therefore, the choice of "which is better" comes down entirely to the deployment environment.

4.1 When AES-128 (or AES-256) is Superior

AES is the superior choice for **Enterprise, Server, and Desktop environments.**

- **Hardware Availability:** Almost every x86/x64 server and modern desktop has AES-NI. In these environments, AES encrypts data with near-zero CPU overhead.
- **Compliance:** AES is a FIPS-approved standard (NIST). For government contracts, financial institutions, and healthcare (HIPAA), AES is legally mandated.
- **Use Cases:** Hard drive encryption (BitLocker, LUKS), enterprise VPNs, and backend HTTPS server infrastructure.

4.2 When ChaCha20 is Superior

ChaCha20 is overwhelmingly better for **Mobile, IoT, and Legacy hardware.**

- **No Dedicated Silicon:** Cheap microcontrollers (ESP32, Arduino), older smartphones, and IoT devices do not have AES-NI. Running AES in software drains batteries quickly. ChaCha20 provides massive speed improvements and battery savings on these devices.
- **Security in Software:** Because the C implementation of ChaCha does not use memory lookups, it is naturally immune to cache-timing attacks, making it far safer than software-AES on shared cloud infrastructure.
- **Use Cases:** Android TLS connections, IoT smart home protocols, WireGuard VPN, and embedded environments where device power constraints are strictly limited.

5. Conclusion & Final Verdict

The implementations of AES-128 and ChaCha represent two fundamentally different philosophies in cryptographic engineering.

AES is an elegant, mathematically complex cipher based on finite fields. It relies on the assumption that hardware manufacturers will build dedicated silicon to support its weight—an assumption that has proven true for enterprise computing. When hardware acceleration is available, **AES is untouchable in efficiency.**

ChaCha, conversely, is a pragmatic, software-first cipher. By restricting its mathematics to simple Addition, Rotation, and XOR operations, it speaks the native language of the CPU. It is **objectively superior** in any environment where cryptography must be executed purely in software.

Ultimately, modern protocols (like TLS 1.3) implement *both*. The server and client perform a handshake to check for hardware capabilities: if AES-NI is detected, they use AES; if they are communicating with a mobile device lacking acceleration, they dynamically fall back to ChaCha20.

References

[1] National Institute of Standards and Technology (NIST). (2001). *Federal Information Processing Standards Publication 197: Advanced Encryption Standard (AES)*. U.S. Department of Commerce.
Available at: https://csrc.nist.gov/publications/detail/fips/197/final

[2] Bernstein, D. J. (2008). *ChaCha, a variant of Salsa20*. The University of Illinois at Chicago. Document ID: 20080128.
Available at: https://cr.yp.to/chacha/chacha-20080128.pdf

[3] Nir, Y., & Langley, A. (2018). *RFC 8439: ChaCha20 and Poly1305 for IETF Protocols*. Internet Engineering Task Force (IETF).
Available at: https://datatracker.ietf.org/doc/html/rfc8439

[4] Google Gemini AI. (2026). *Conversational Analysis and Low-Level C Implementation Architecture: AES-128 vs. ChaCha20*. Generated via interactive computational prompting, August 2026.
Available at: https://gemini.google.com